

AI Agent Evaluation — Complete Consolidated Guide

1. Overview

AI agent evaluation is the process of measuring how well an agent reasons, selects and calls tools, and completes tasks—at the scope that matches each failure mode—so you can pinpoint exactly what is broken.

A successful agent outcome depends on both:

1. **Reasoning**
2. **Action**

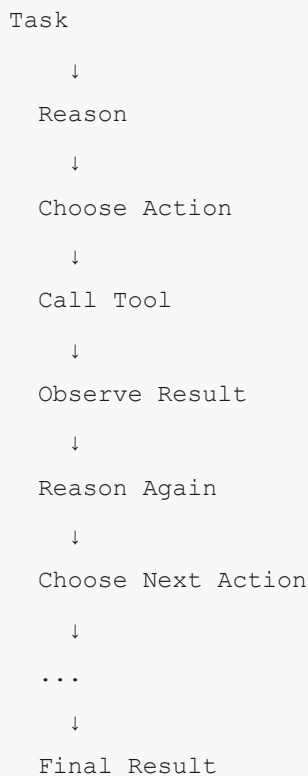
Because an agent can fail at different points, evaluation should not be treated as a single score. Different evaluation scopes and metrics inspect different parts of the agent's behavior.

2. The AI Agent Architecture

An AI agent can be viewed as two interacting layers:



The reasoning and action layers form a loop:



3. Three Complementary Evaluation Scopes

AI agent evaluation uses three major scopes.

Scope	What it evaluates	Main question
End-to-End / Black-Box	Observable input and final output	Did the agent produce the desired result?
Trajectory-Based	Complete ordered execution trace	How did the agent reach the result?
Component-Level / Step-Wise	One span or decision in isolation	Was this particular decision correct?

3.1 End-to-End / Black-Box Evaluation

End-to-end evaluation treats the agent as a black box.

Input

↓

Agent

↓

Final Output

The internal execution is not inspected.

Best used for

- Overall behavior
- Final answer quality
- Final task success
- High-level regression testing

Limitation

If the final result is wrong, end-to-end evaluation may tell you **that it failed**, but not necessarily **where it failed**.

3.2 Trajectory-Based Evaluation

Trajectory-based evaluation examines the complete ordered execution trace.

A trajectory can contain:

```
User Task
↓
Reasoning / Plan
↓
LLM Call
↓
Tool Call
↓
Tool Arguments
↓
Tool Output / Observation
↓
Next LLM Call
↓
Next Tool Call
↓
...
↓
Final Output
```

Best used for

- Planning
- Plan adherence
- Task completion
- Step efficiency
- Understanding why an agent failed

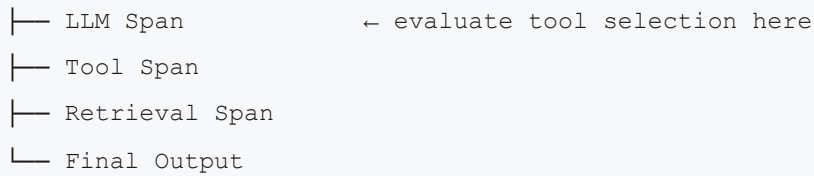
Trajectory-based metrics require tracing because they inspect the complete ordered execution.

3.3 Component-Level / Step-Wise Evaluation

Component-level evaluation focuses on one specific span or decision.

For example:

Agent



Best used for

- Tool selection
- Tool arguments
- Individual LLM decisions
- Specific agent components

The important principle is:

Evaluate each failure mode at the scope where the relevant information is visible.

4. Reasoning Layer

The **reasoning layer** contains the LLM.

Its responsibilities include:

- Understanding the user task
- Creating a plan
- Deciding what should happen next
- Selecting tools
- Adapting to observations

The supplied evaluation framework identifies two trajectory metrics for reasoning and planning:

- `PlanQualityMetric`
- `PlanAdherenceMetric`

5. Plan Quality

5.1 What is Plan Quality?

`PlanQualityMetric` evaluates whether the plan generated by the agent is logical, complete, and aligned with the task.

The important question is:

Is the strategy itself good?

A plan can be evaluated independently from whether the agent follows it.

5.2 How Plan Quality Is Calculated

The supplied description explains the process as:

Step 1 — Extract the Task

The task is extracted from the trace.

The task represents the user's goal or intent and should be actionable.

Step 2 — Extract the Plan

The plan is extracted from the agent's `thinking` or `reasoning`.

If there are no statements that clearly define or imply a plan in the trace, the supplied description states that the metric passes by default with a score of `1`.

Step 3 — Calculate Alignment

Conceptually:

```
Plan Quality Score
=
AlignmentScore(Task, Plan)
```

The alignment score uses an LLM to generate the final score from the task and extracted plan.

5.3 Example

User Task

Find AI/ML Engineer jobs in Hyderabad.

Good Plan

1. Understand role = AI/ML Engineer
2. Understand location = Hyderabad
3. Search relevant jobs
4. Filter results by location
5. Rank relevant jobs
6. Return recommendations

Poor Plan

1. Search jobs
2. Generate a cover letter
3. Search unrelated information
4. Search jobs again
5. Return results

The second plan contains unnecessary actions and is poorly aligned with the actual task.

6. Plan Adherence

6.1 What is Plan Adherence?

`PlanAdherenceMetric` evaluates whether the agent follows its own plan during execution. The important question is:

Did the agent actually execute the strategy it planned?

An agent can have:

- A good plan but poor adherence
- A poor plan but strong adherence

Therefore, Plan Quality and Plan Adherence are different metrics.

6.2 How Plan Adherence Is Calculated

The supplied description explains:

Step 1 — Extract the Task

The user's task is extracted from the trace.

Step 2 — Extract the Plan

The plan is extracted from the trace's reasoning/thinking.

Step 3 — Evaluate Execution Steps

The actual execution steps are compared against the task and plan.

Conceptually:

```
Plan Adherence Score
=
AlignmentScore((Task, Plan), Execution Steps)
```

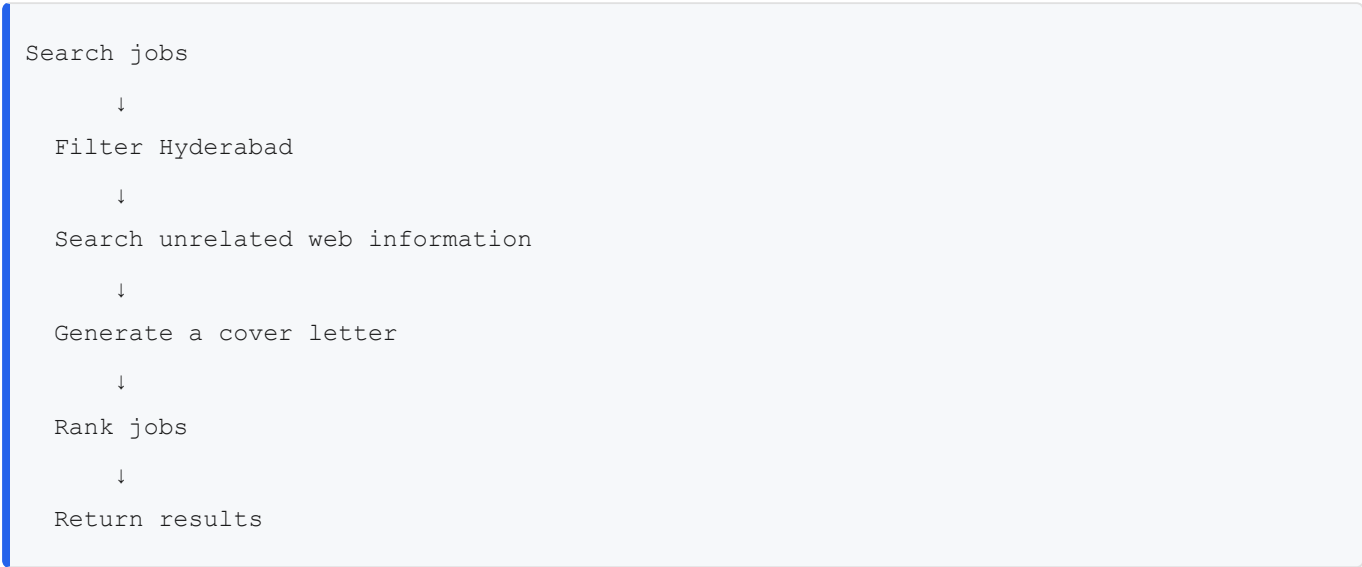
The final alignment score is generated using an LLM.

6.3 Example

Plan

```
Search jobs
  ↓
Filter Hyderabad
  ↓
Rank jobs
  ↓
Return results
```

Actual Execution



The agent deviated from its planned strategy.

7. Plan Quality vs Plan Adherence

Situation	Plan Quality	Plan Adherence	Meaning
Good plan + followed	High	High	Ideal
Bad plan + followed	Low	High	Strategy is poor, execution is consistent
Good plan + not followed	High	Low	Strategy is good, execution deviated
Bad plan + not followed	Low	Low	Both planning and execution need improvement

This distinction is useful when diagnosing whether a problem is in:

- The reasoning strategy
- The execution/control logic

8. Action Layer

The **action layer** contains tools such as:

- Function calls
- APIs
- Search tools
- Retrievers

- Databases
- External services
- Browser tools
- Other executable actions

The action layer is primarily evaluated using:

- `ToolCorrectnessMetric`
- `ArgumentCorrectnessMetric`

9. Tool Correctness

9.1 What is Tool Correctness?

`ToolCorrectnessMetric` evaluates an LLM agent's function/tool calling ability.

It checks whether:

1. The tools that were expected to be used were actually called.
2. The tools selected by the agent were appropriate/optimal for the task.

The supplied documentation describes this as an agentic LLM metric.

9.2 Required Test Case Fields

To use `ToolCorrectnessMetric`, the test case requires:

- `input`
- `actual_output`
- `tools_called`
- `expected_tools`

Conceptually:

```
Input
+
Actual Output
+
Tools Called
+
Expected Tools
```

9.3 Example

```
from deepeval.test_case import LLMTestCase, ToolCall
from deepeval.metrics import ToolCorrectnessMetric
from deepeval import evaluate

test_case = LLMTestCase(
    input="What if these shoes don't fit?",
    actual_output="We offer a 30-day full refund at no extra cost.",
    tools_called=[
        ToolCall(name="WebSearch"),
        ToolCall(name="ToolQuery")
    ],
    expected_tools=[
        ToolCall(name="WebSearch")
    ]
)

metric = ToolCorrectnessMetric()

evaluate(
    test_cases=[test_case],
    metrics=[metric]
)
```

10. Tool Correctness Configuration

The supplied documentation describes several optional controls.

available_tools

A list of tools that were available to the LLM agent.

It provides context for evaluating tool-selection optimality.

threshold

Minimum passing threshold.

The supplied default is:

```
0.5
```

It can also be set to `None` for score-only mode.

`evaluation_params`

Controls how strict correctness should be.

The supplied options include:

- `ToolCallParams.INPUT_PARAMETERS`
- `ToolCallParams.OUTPUT`

This allows evaluation beyond just matching the tool name.

`include_reason`

Controls whether an evaluation reason is included.

Default:

```
True
```

`strict_mode`

When enabled:

```
1 = perfect  
0 = otherwise
```

It also overrides the threshold and sets it to `1`.

`verbose_mode`

When enabled, intermediate calculation steps are printed.

`should_consider_ordering`

When enabled, the order of tool calls is considered.

For example:

```
Expected:  
  WebSearch  
  ToolQuery  
  WebSearch
```

versus:

```
Actual:  
  WebSearch  
  WebSearch  
  ToolQuery
```

If ordering is enabled, the sequence matters.

`should_exact_match`

When enabled, `tools_called` and `expected_tools` must match exactly.

It is stricter than ordering and overrides the effect of `should_consider_ordering`.

11. Tool Correctness Calculation

The supplied documentation describes a deterministic score based on tool usage.

Conceptually:

```
Tool Correctness  
=  
  Correctly Used Tools  
/  
  Total Tools Called
```

More specifically, correctness can consider:

- Tool names
- Input parameters, if configured
- Outputs, if configured
- Ordering
- Exact matching

11.1 Deterministic + Model-Based Evaluation

The supplied documentation states that Tool Correctness is different from most metrics because it can combine deterministic and non-deterministic evaluation.

Step 1 — Deterministic Evaluation

Compare:

```
tools_called
vs
expected_tools
```

Step 2 — Optional Optimality Evaluation

If `available_tools` is supplied, an LLM evaluates whether the selected tools were the most optimal for the task.

Final Score

The supplied description states that the final score is the **minimum** of:

```
Deterministic Score
LLM-Based Optimality Score
```

If `available_tools` is not supplied, the additional model-based optimality evaluation does not occur.

12. Tool Correctness Evaluation Modes

The supplied material documents three modes:

```
llm
  hybrid
  system_one
```

LLM Mode

The LLM evaluates the relevant model-based selection judgment.

This is the default mode in the supplied documentation.

Hybrid Mode

- LLM extracts information
- Jev / System One evaluates the tool-selection judgment

If the Jev call fails, the LLM can perform the decision instead.

System One Mode

Jev evaluates the tool-selection judgment without an LLM judge.

13. Tool Correctness Within Components

`ToolCorrectnessMetric` can also be used for component-level evaluation.

A component can be observed and the metric attached to that span.

Conceptually:

```
@observe(metrics=[metric])
def inner_component():
    ...
```

The test case can then be created at runtime and attached to the current span.

This is useful when the goal is specifically to evaluate an LLM component responsible for tool selection.

14. Tool Correctness vs Argument Correctness

These metrics solve different problems.

Metric	Question
Tool Correctness	Did the agent choose the right tool?
Argument Correctness	Did the agent provide correct arguments to the tool?

Example:

```
Task:
  Find AI/ML Engineer jobs in Hyderabad.

Selected Tool:
  LinkedInJobSearch
```

The tool selection can be correct.

But:

```
{
  "role": "Data Scientist",
  "location": "Delhi"
}
```

can still be incorrect.

Therefore:

```
Tool Correctness      → High
Argument Correctness  → Low
```

This distinction is one of the most important concepts in agent evaluation.

15. Argument Correctness

15.1 What is Argument Correctness?

`ArgumentCorrectnessMetric` evaluates whether the arguments generated for each tool call are correct based on the input/task.

It is:

- LLM-judged
- Referenceless

The supplied documentation explicitly notes that deterministic argument evaluation can instead be done using Tool Correctness with input-parameter matching.

15.2 Required Test Case Fields

The required fields are:

- `input`
- `actual_output`
- `tools_called`

Unlike Tool Correctness, an `expected_tools` list is not required.

15.3 Example

```
from deepeval.metrics import ArgumentCorrectnessMetric
from deepeval.test_case import LLMTestCase, ToolCall
from deepeval import evaluate

metric = ArgumentCorrectnessMetric(
    threshold=0.7,
    model="gpt-4",
    include_reason=True
)

test_case = LLMTestCase(
    input="When did Trump first raise tariffs?",
    actual_output="Trump first raised tariffs in 2018 during the U.S.-China trade war.",
    tools_called=[
        ToolCall(
            name="WebSearch Tool",
            description="Tool to search for information on the web.",
            input={
                "search_query": "Trump first raised tariffs year"
            }
        ),
        ToolCall(
            name="History FunFact Tool",
            description="Tool to provide a fun fact about the topic.",
            input={
                "topic": "Trump tariffs"
            }
        )
    ]
)

evaluate(
    test_cases=[test_case],
    metrics=[metric]
)
```

16. Argument Correctness Calculation

The supplied documentation gives the conceptual equation:

```
Argument Correctness
=
Correctly Generated Input Parameters
/
Total Number of Tool Calls
```

The LLM determines whether each tool-call argument correctly addresses the input/task.

17. Argument Correctness Configuration

The supplied documentation describes parameters including:

- `threshold`
- `model`
- `include_reason`
- `strict_mode`
- `async_mode`
- `verbose_mode`
- `flaky`
- `system_one_model`
- `eval_mode`

Default threshold

```
0.5
```

Strict mode

With `strict_mode=True`:

```
1 = perfect
0 = otherwise
```

Custom evaluator model

The `model` can specify the LLM used for the evaluation, including a supported custom model type.

18. Argument Correctness Within Components

Argument Correctness can be attached to a nested component using tracing.

Conceptually:

```
@observe(metrics=[metric])
def inner_component():
    test_case = LLMTestCase(
        input="...",
        actual_output="...",
        tools_called=[...]
    )

    update_current_span(
        test_case=test_case
    )
```

This makes it possible to evaluate the component responsible for generating tool calls and their arguments.

19. Argument Correctness Evaluation Modes

The supplied documentation also describes:

```
llm
  hybrid
    system_one
```

LLM Mode

The LLM evaluates argument correctness.

Hybrid Mode

Jev / System One can evaluate individual tool-call argument correctness.

The supplied description says the assessment can become a yes/no judgment for each tool call, with:

```
P(yes) >= 0.5
```

counted as correct.

System One Mode

Jev evaluates the complete metric using the structured tool-call information.

The supplied documentation describes three questions covering:

1. Whether every tool call has arguments that correctly address the task
2. Whether any tool call contains missing, irrelevant, or wrong arguments
3. How many tool calls have correctly addressed arguments

These answers are converted into values in `[0, 1]` and combined using a weighted mean.

20. Overall Execution Evaluation

Once tool usage and reasoning are evaluated, the complete execution can be evaluated using:

- `TaskCompletionMetric`
- `StepEfficiencyMetric`

These are trajectory-level metrics.

21. Task Completion

21.1 What is Task Completion?

`TaskCompletionMetric` evaluates how effectively an LLM agent accomplishes a task. It analyzes the agent's full trace to determine whether the intended task was successfully completed.

The supplied documentation explicitly states:

```
Task Completion analyzes the agent's full trace to determine task success.
```

Therefore, tracing is required for trajectory-based use.

22. Task Completion Usage

The supplied example uses:

```
from deepeval.dataset import Golden, EvaluationDataset
from deepeval.metrics import TaskCompletionMetric
from deepeval.tracing import observe

@observe()
def trip_planner_agent(input):
    ...

dataset = EvaluationDataset(
    goldens=[
        Golden(input="This is a test query")
    ]
)

task_completion = TaskCompletionMetric(
    threshold=0.7,
    model="gpt-4o"
)

for golden in dataset.eval_iterator(
    metrics=[task_completion]
):
    trip_planner_agent(golden.input)
```

23. Task Completion Parameters

The supplied material describes parameters including:

- `threshold`
- `task`
- `model`
- `include_reason`
- `strict_mode`
- `async_mode`
- `verbose_mode`
- `flaky`

- `system_one_model`
- `eval_mode`

task

The task can optionally be supplied directly.

If it is not supplied:

```
task = None
```

the task is inferred from the trace.

24. Task Completion Calculation

The supplied equation is:

```
Task Completion Score
=
AlignmentScore(Task, Outcome)
```

Step 1

Extract:

```
Task
Outcome
```

from the trace.

Step 2

Calculate how well the outcome aligns with the task.

The alignment score is generated by an LLM in LLM-as-a-judge mode.

25. Task Completion Example

User Task

Find three AI/ML jobs in Hyderabad.

Successful run

```
Understand request
↓
Search jobs
↓
Find 3 relevant jobs
↓
Return them
```

Result:

Task Completion → High

Partial run

```
Understand request
↓
Search jobs
↓
Find jobs
↓
Stop without returning them
```

Result:

Task Completion → Low / Partial

26. Task Completion vs Plan Adherence

These answer different questions.

Metric	Question
Task Completion	Did the agent accomplish the task?
Plan Adherence	Did the agent follow its own plan?

An agent can:

```
Follow its plan but fail the task
```

or:

```
Ignore its plan but still complete the task
```

Therefore, these metrics should not be treated as interchangeable.

27. Task Completion Evaluation Modes

The supplied documentation describes:

```
llm
  hybrid
  system_one
```

LLM

LLM extracts task/outcome and judges alignment.

Hybrid

- LLM extracts task/outcome
- Jev provides the alignment score

System One

Jev evaluates the trace directly.

The supplied documentation describes structured questions concerning:

1. Whether the run fully accomplishes the user's task
2. Whether the final outcome matches the requested task
3. How completely the task was accomplished

These values are combined into a weighted mean.

28. Step Efficiency

28.1 What is Step Efficiency?

`StepEfficiencyMetric` evaluates how efficiently the agent executes a task. It analyzes the full trace to determine:

- What the task was
- What execution steps occurred
- Whether those steps were necessary
- Whether the route contained unnecessary actions

The supplied material explicitly states that Step Efficiency is a trace-only metric. It must be used through:

- `evals_iterator`
- or `observe`

It cannot be used as a standalone metric.

29. Step Efficiency Usage

The supplied example uses:

```

from deepeval.tracing import observe, update_current_trace

from deepeval.dataset import Golden, EvaluationDataset
from deepeval.metrics import StepEfficiencyMetric
from deepeval.test_case import ToolCall


@observe
def tool_call(input):
    ...


@observe
def agent(input):
    tools = tool_call(input)
    output = llm(input, tools)

    update_current_trace(
        input=input,
        output=output,
        tools_called=tools
    )

    return output


dataset = EvaluationDataset(
    goldens=[
        Golden(
            input="What's the weather like in SF?"
        )
    ]
)


metric = StepEfficiencyMetric(
    threshold=0.7,
    model="gpt-4o"
)


for golden in dataset.evals_iterator(
    metrics=[metric]
):
    agent(golden.input)

```

30. Step Efficiency Calculation

The supplied description explains:

Step 1 — Extract the Task

The task is extracted from the trace.

Step 2 — Evaluate Execution Steps

The agent's execution steps are evaluated for efficiency.

Conceptually:

```
Step Efficiency Score
=
AlignmentScore(Task, Execution Steps)
```

The metric penalizes actions that are not strictly required to complete the task.

Examples include:

- Duplicate actions
- Repeated searches
- Unnecessary retrieval
- Detours
- Loops
- Speculative actions

31. Efficient vs Inefficient Execution

Efficient

```
Search
↓
Filter
↓
Rank
↓
Return
```

Inefficient

```
Search
↓
Search again
↓
Search again
↓
Retrieve unnecessary information
↓
Search again
↓
Filter
↓
Rank
↓
Return
```

Both could potentially produce a correct final answer.

But:

```
Task Completion
```

could be high for both, while:

```
Step Efficiency
```

would be lower for the second.

32. Task Completion vs Step Efficiency

This distinction is critical.

Run	Task Completion	Step Efficiency
Fast and successful	High	High
Slow/bloated but successful	High	Low
Efficient but unsuccessful	Low	High
Inefficient and unsuccessful	Low	Low

Key principle

*Task Completion evaluates the **outcome**.*

*Step Efficiency evaluates the **path**.*

The supplied documentation recommends evaluating both because one does not guarantee the other.

33. Step Efficiency Evaluation Modes

The supplied documentation describes:

```
llm
  hybrid
  system_one
```

LLM

The LLM extracts the task and evaluates execution efficiency.

Hybrid

- LLM extracts task information
- Jev evaluates efficiency

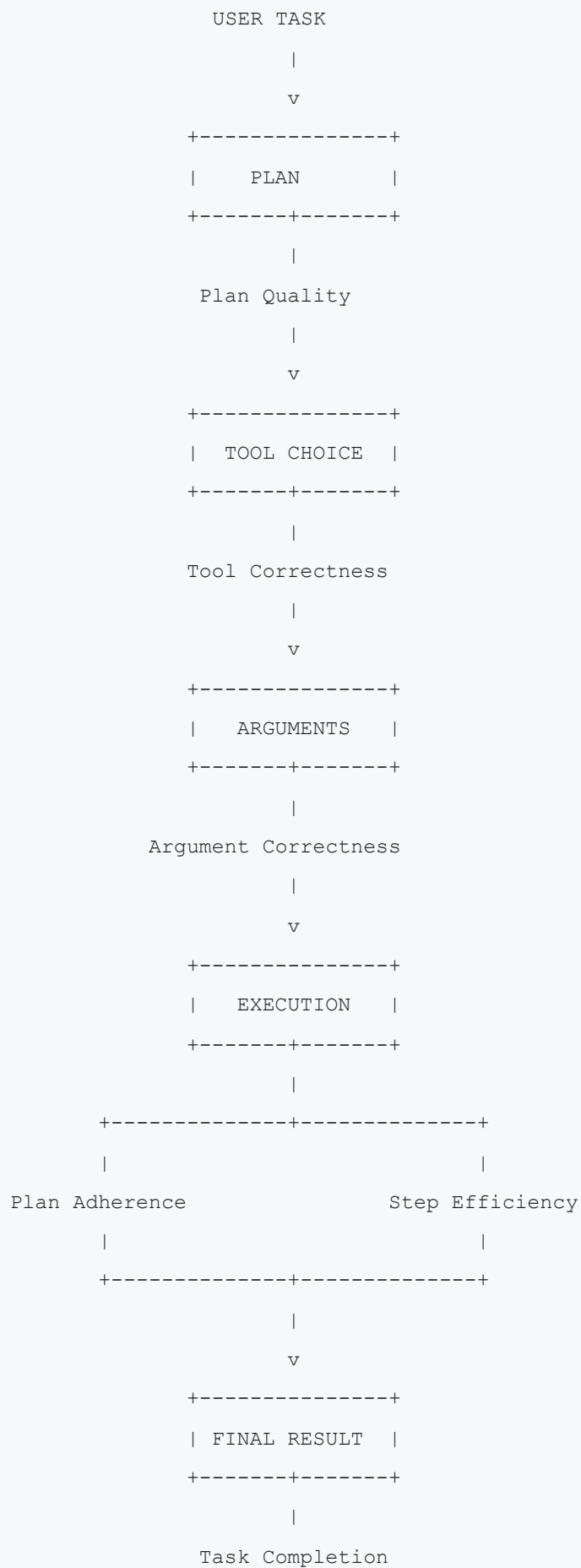
System One

Jev directly evaluates the trace and considers:

1. Whether every step was strictly necessary
2. Whether the path was direct
3. Overall efficiency based on the number/directness of steps

34. Full Agent Evaluation Map

A practical way to think about all metrics:



35. Complete Metric Matrix

Metric	Layer	Scope	Main Question	Trace Required?
PlanQualityMetric	Reasoning	Trajectory	Is the plan good?	Yes / trajectory
PlanAdherenceMetric	Reasoning	Trajectory	Was the plan followed?	Yes / trajectory
ToolCorrectnessMetric	Action / LLM decision	Component or E2E	Was the right tool selected?	Trace for component-level
ArgumentCorrectnessMetric	Action / LLM decision	Component or E2E	Were the arguments correct?	Trace optional for component-level
TaskCompletionMetric	Overall execution	Trajectory	Was the task completed?	Yes
StepEfficiencyMetric	Overall execution	Trajectory	Was the path efficient?	Yes

36. Dataset-Based Evaluation

A repeatable evaluation system should use an evaluation dataset.

The supplied examples use:

```
EvaluationDataset
+
Golden
```

Conceptually:

```
dataset = EvaluationDataset(
    goldens=[
        Golden(input="Task 1"),
        Golden(input="Task 2"),
        Golden(input="Task 3")
    ]
)
```

Then the agent is run against the dataset.

37. Example Evaluation Dataset

For an AI Resume Agent, a dataset might contain:

```
Golden 1:
  Find AI/ML Engineer jobs in Hyderabad.

Golden 2:
  Tailor my resume for this job description.

Golden 3:
  Analyze my resume for ATS compatibility.

Golden 4:
  Create a cover letter for this job.

Golden 5:
  Conduct a mock interview.

Golden 6:
  Generate a career roadmap.

Golden 7:
  Find suitable job roles based on my profile.
```

The same dataset can be run repeatedly after changes.

38. Why a Dataset Matters

A dataset makes evaluation repeatable.

Without a dataset:

```
Change code
↓
Try a few examples manually
↓
Assume improvement
```

With a dataset:

Baseline

↓

Run dataset

↓

Record scores

↓

Change system

↓

Run same dataset

↓

Compare scores

This enables regression testing.

39. Recommended Evaluation Workflow

Step 1 — Create Evaluation Cases

Prepare realistic tasks.

Include:

- Normal cases
- Edge cases
- Ambiguous cases
- Tool-selection cases
- Failure-prone cases

Step 2 — Run the Real Agent

Run the actual production-like agent.

Do not rely only on manually constructed hypothetical traces when measuring real behavior.

Step 3 — Capture the Trace

Capture:

User Input

↓

Agent Span

↓

LLM Span

↓

Plan / Reasoning

↓

Tool Call

↓

Tool Arguments

↓

Tool Result

↓

Next LLM Call

↓

Next Tool Call

↓

Final Output

Step 4 — Evaluate Planning

Use:

Plan Quality

Plan Adherence

Step 5 — Evaluate Tool Selection

Use:

Tool Correctness

Step 6 — Evaluate Tool Arguments

Use:

Argument Correctness

Step 7 — Evaluate Final Outcome

Use:

Task Completion

Step 8 — Evaluate Execution Efficiency

Use:

Step Efficiency

Step 9 — Diagnose the Failure

Map the low metric back to the responsible part of the system.

Step 10 — Fix and Re-Evaluate

Potential areas to change:

- System prompt
- Agent prompt
- Tool description
- Tool schema
- Routing logic
- Planning logic
- Model
- Retrieval

- Retry logic
- Stop conditions
- Application code

Then run the same evaluation dataset again.

40. Failure Diagnosis Matrix

Observed Problem	Metric	Likely Failure Area
Final task is not achieved	Task Completion	Overall agent behavior
Wrong tool is selected	Tool Correctness	Tool selection / reasoning
Correct tool, wrong parameters	Argument Correctness	Argument generation
Plan is poor	Plan Quality	Planning / reasoning
Plan is good but ignored	Plan Adherence	Execution / control logic
Too many or repeated steps	Step Efficiency	Agent loop / routing / stopping
Task succeeds but execution is bloated	Step Efficiency	Execution strategy
Tool is right but inputs are wrong	Argument Correctness	LLM tool-call generation

41. Example: Job Recommendation Agent

Consider an agent with the task:

Find AI/ML Engineer jobs in Hyderabad and recommend the best matches.

Expected Plan

1. Understand role
2. Understand location
3. Search relevant jobs
4. Filter by location
5. Rank jobs
6. Return recommendations

Expected Tool

LinkedInJobSearch

Expected Arguments

```
{  
  "role": "AI/ML Engineer",  
  "location": "Hyderabad"  
}
```

Expected Execution

```
Search  
↓  
Filter  
↓  
Rank  
↓  
Return
```

Expected Outcome

A set of relevant job recommendations is returned.

42. Evaluation of the Job Recommendation Agent

Agent Part	Evaluation
Plan is sensible	Plan Quality
Plan is followed	Plan Adherence
LinkedInJobSearch is selected	Tool Correctness
Role/location are correct	Argument Correctness
No unnecessary repeated searches	Step Efficiency
Recommendations are returned	Task Completion

This demonstrates why multiple metrics are needed.
A single final score would not tell you whether a failure came from:

- Planning
- Tool selection
- Arguments
- Execution
- Outcome

43. Example Failure Scenarios

Scenario 1 — Wrong Tool

Task:

Find jobs in Hyderabad.

Agent:

Calls WeatherSearch

Diagnosis:

Tool Correctness → Low

Scenario 2 — Correct Tool, Wrong Arguments

Task:

Find AI/ML Engineer jobs in Hyderabad.

Agent:

Calls LinkedInJobSearch

Arguments:

role = Data Scientist

location = Delhi

Diagnosis:

Tool Correctness → High

Argument Correctness → Low

Scenario 3 — Poor Plan

Task:

Find jobs.

Plan:

Search jobs

Generate a cover letter

Analyze weather

Search jobs again

Diagnosis:

Plan Quality → Low

Scenario 4 — Good Plan, Deviates During Execution

Plan:

Search → Filter → Rank → Return

Execution:

Search → Filter → Search unrelated web → Generate cover letter → Rank → Return

Diagnosis:

Plan Quality → High

Plan Adherence → Low

Scenario 5 — Task Completed Inefficiently

Search

↓

Search again

↓

Search again

↓

Retrieve unnecessary data

↓

Filter

↓

Rank

↓

Return

Diagnosis:

Task Completion → High

Step Efficiency → Low

Scenario 6 — Efficient but Incomplete

Search

↓

Filter

↓

Stop

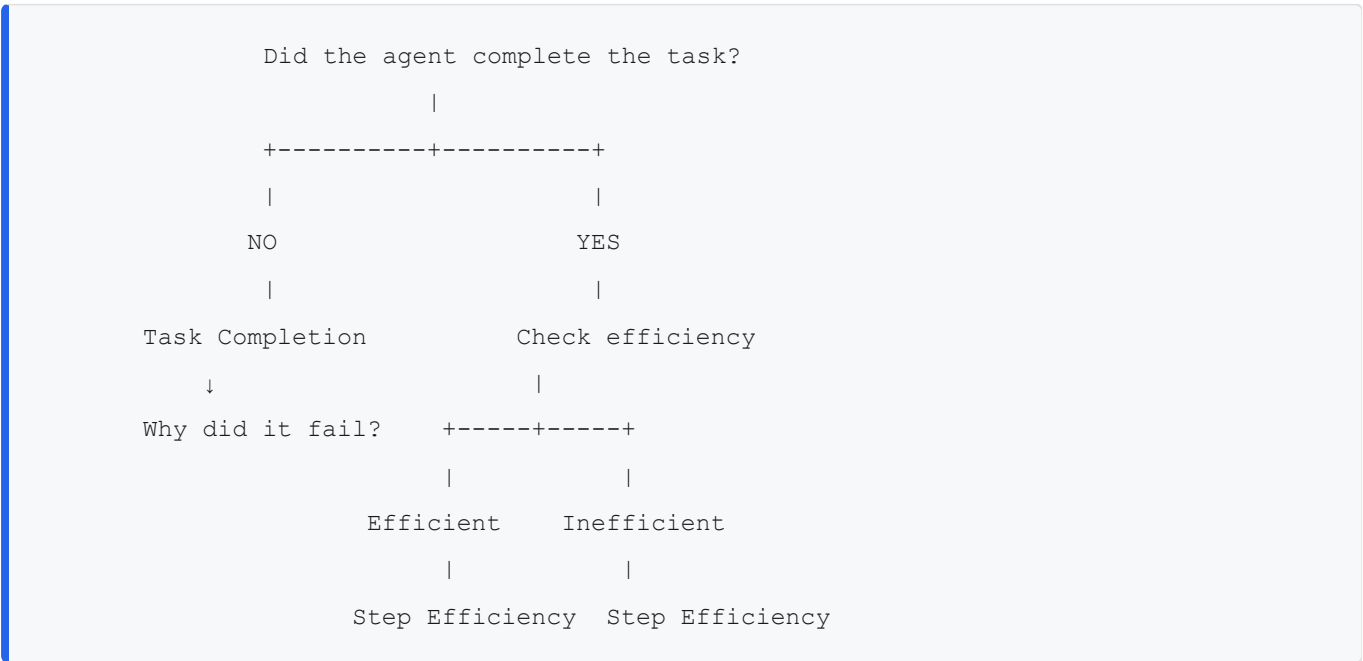
Diagnosis:

Task Completion → Low

Step Efficiency → Potentially High

44. Evaluation Decision Tree

Use the following simple decision process:



For the cause of failure:

Was the plan bad?

↓

Plan Quality

Was the plan ignored?

↓

Plan Adherence

Was the wrong tool selected?

↓

Tool Correctness

Were tool arguments wrong?

↓

Argument Correctness

45. Evaluation Modes

The supplied DeepEval material documents three evaluation modes:

```
llm
  hybrid
  system_one
```

45.1 LLM-as-a-Judge

The LLM evaluates the relevant criterion.

Example:

```
deepeval set-eval-mode llm
```

This is described as the default mode in the supplied documentation.

45.2 Hybrid

The LLM performs extraction while Jev / System One performs the scoring decision.

```
deepeval set-eval-mode hybrid
```

This approach is used in the supplied metric documentation for several of the metrics.

45.3 System One / Jev-as-a-Judge

Jev performs the evaluation without an LLM judge.

```
deepeval set-eval-mode system_one
```

The supplied documentation notes that scoring procedures differ between evaluation modes, so when comparing scores across runs, it is important to keep the evaluation mode consistent.

46. Thresholds

Many DeepEval metrics use a threshold to determine whether a score passes.

Conceptually:

```
Score >= Threshold
      ↓
    Pass
```

```
Score < Threshold
      ↓
    Fail
```

The supplied metric pages commonly show:

```
threshold = 0.5
```

as the default for several metrics.

Examples in the supplied documentation also use:

```
threshold = 0.7
```

for stricter examples.

Thresholds should therefore be chosen deliberately for the application's requirements.

47. Score-Only Mode

The supplied documentation notes that some thresholds can be set to:

```
None
```

to run the metric in score-only mode.

This is useful when the goal is to observe metric values rather than convert them into pass/fail decisions.

48. Strict Mode

When `strict_mode=True`, the supplied metric documentation describes a binary interpretation:

```
1 = perfect  
0 = not perfect
```

Strict mode also overrides the threshold and sets it to `1`.

49. Include Reason

When available,:

```
include_reason=True
```

provides an explanation for the score.

Reasons are valuable during debugging because the score tells you **how much** the agent succeeded, while the reason can explain **why**.

50. Verbose Mode

When:

```
verbose_mode=True
```

the supplied metric documentation states that intermediate calculation steps can be printed.

This is particularly useful when debugging unexpected scores.

51. Standalone vs Evaluation Pipeline

Some metrics can be measured directly:

```
metric.measure(test_case)
```

and then:

```
print(metric.score)
print(metric.reason)
```

However, the supplied documentation warns that standalone measurement does not provide all the benefits of the full evaluation pipeline, such as:

- Testing reports
 - Confident AI platform benefits
 - Optimizations
 - Speed
 - Caching
 - Computation optimizations
-

52. Trace-Only Metrics

The supplied material explicitly identifies Step Efficiency as a trace-only metric.

It must be attached to:

- An observed trace
- `evals_iterator`

- Or the appropriate tracing mechanism

The same trajectory-based principle applies to Task Completion when evaluating the complete run.

53. Component-Level Evaluation Example

A component can be independently evaluated.

Conceptually:

```
@observe(metrics=[metric])
def inner_component():
    test_case = LLMTestCase(
        input="...",
        actual_output="...",
        tools_called=[...]
    )

    update_current_span(
        test_case=test_case
    )
```

Then:

```
@observe
def llm_app(input):
    inner_component()
    return ...
```

This architecture allows evaluation at a specific point in the agent execution.

54. Evaluation of a Tool-Calling LLM Component

Suppose your agent looks like:

```
User
  ↓
LLM
  ↓
Tool Decision
  ↓
Tool Call
```

The component responsible for:

```
LLM → Tool Decision
```

is exactly where Tool Correctness and Argument Correctness become useful at component level.

The reason is that this is the point where the model:

- Chooses the tool
- Generates the arguments

55. Full Agent Trace Example

A complete trace might look like:

```
ROOT AGENT
|
|-- Input:
|   "Find AI/ML jobs in Hyderabad"
|
|-- LLM
|   |
|   |-- Plan:
|   |   Search jobs
|   |   Filter Hyderabad
|   |   Rank results
|   |
|   |-- Tool Call:
|   |   LinkedInJobSearch
|   |   arguments={
|   |       role="AI/ML Engineer",
|   |       location="Hyderabad"
|   |   }
|   |
|-- Tool Output
|   |
|   |-- 30 job results
|   |
|-- LLM
|   |
|   |-- Filter
|   |-- Rank
|   |
|-- Final Output
|   |
|   |-- 5 recommendations
```

This single trace provides evidence for multiple metrics.

56. How One Trace Supports Multiple Metrics

From the same trace:

Plan Quality

Look at:

Plan

Plan Adherence

Compare:

Plan

vs

Actual Execution

Tool Correctness

Look at:

Tool Calls

vs

Expected Tools

Argument Correctness

Look at:

Tool Arguments

Step Efficiency

Look at:

All Execution Steps

Task Completion

Look at:

Final Outcome

This is why tracing is central to agent evaluation.

57. Recommended Test Design

A strong evaluation dataset should contain different categories.

Normal Cases

Typical user requests.

Tool-Selection Cases

Tasks where multiple tools exist but only one is appropriate.

Argument Cases

Tasks where correct tool arguments matter.

Planning Cases

Tasks requiring multiple logical steps.

Efficiency Cases

Tasks where unnecessary calls can easily occur.

Failure Cases

Tasks designed to trigger common agent weaknesses.

Edge Cases

Unusual or ambiguous inputs.

58. Example Dataset Categories

Category: Tool Selection

"Find current AI jobs in Hyderabad."

Category: Argument Generation

"Find AI/ML Engineer jobs in Hyderabad with recent postings."

Category: Planning

"Analyze my resume and suggest suitable roles."

Category: Task Completion

"Tailor my resume to this job description."

Category: Efficiency

"Find the top 5 matching jobs from a large list."

59. Regression Evaluation

Whenever you change:

- Model
- Prompt
- Tool schema
- Routing
- Agent graph
- Retrieval
- Memory
- Retry logic
- Stopping logic

run the evaluation dataset again.

Conceptually:

```
BEFORE CHANGE
|
v
Run Evaluation
|
v
Record Metrics
|
v
Change System
|
v
Run Again
|
v
Compare Results
```

60. What a Good Agent Should Achieve

A strong agent should ideally have:

```
Good Plan
+
Plan Followed
+
Correct Tool
+
Correct Arguments
+
Efficient Execution
+
Successful Outcome
```

Mapped to metrics:

Good Plan

→ Plan Quality

Plan Followed

→ Plan Adherence

Correct Tool

→ Tool Correctness

Correct Arguments

→ Argument Correctness

Efficient Execution

→ Step Efficiency

Successful Outcome

→ Task Completion

61. One-Line Mental Model

Remember:

PLAN

↓

TOOL

↓

ARGUMENTS

↓

STEPS

↓

OUTCOME

And evaluate:

PLAN → Plan Quality
PLAN USED → Plan Adherence
TOOL → Tool Correctness
ARGUMENTS → Argument Correctness
STEPS → Step Efficiency
OUTCOME → Task Completion

62. Quick Cheat Sheet

Metric	Remember It As
PlanQualityMetric	Was the plan good?
PlanAdherenceMetric	Did it follow the plan?
ToolCorrectnessMetric	Right tool?
ArgumentCorrectnessMetric	Right inputs?
StepEfficiencyMetric	Efficient path?
TaskCompletionMetric	Task finished?

63. Final Practical Checklist

Before declaring an AI agent "good", check:

- ☐ Do I have a representative evaluation dataset?
- ☐ Do my test cases reflect real user tasks?
- ☐ Can I capture the complete agent trace?
- ☐ Can I inspect the plan?
- ☐ Can I inspect tool calls?
- ☐ Can I inspect tool arguments?
- ☐ Can I inspect tool outputs?
- ☐ Can I evaluate whether the plan was followed?
- ☐ Can I evaluate whether the final task was completed?
- ☐ Can I detect redundant steps?
- ☐ Do I have thresholds appropriate to the application?
- ☐ Do I keep the evaluation mode consistent when comparing runs?
- ☐ Do I rerun the same dataset after system changes?

64. Final Summary

AI agent evaluation should be treated as a **diagnostic system**, not just a final-score system.

The key mapping is:

Reasoning

|

+-- Plan Quality

|

+-- Plan Adherence

Action

|

+-- Tool Correctness

|

+-- Argument Correctness

Overall Execution

|

+-- Step Efficiency

|

+-- Task Completion

The main idea is:

Evaluate the part of the agent responsible for the failure.

If the plan is poor, evaluate planning.

If the wrong tool is selected, evaluate tool correctness.

If the right tool receives incorrect parameters, evaluate argument correctness.

If the agent takes unnecessary actions, evaluate step efficiency.

If the user's actual goal is not achieved, evaluate task completion.

For a complete understanding of an AI agent, use multiple complementary metrics rather than relying on one score.

65. Source Basis

This consolidated guide is based on the evaluation framework supplied in the conversation and the supplied DeepEval documentation covering:

- Tool Correctness
- Argument Correctness
- Task Completion
- Step Efficiency

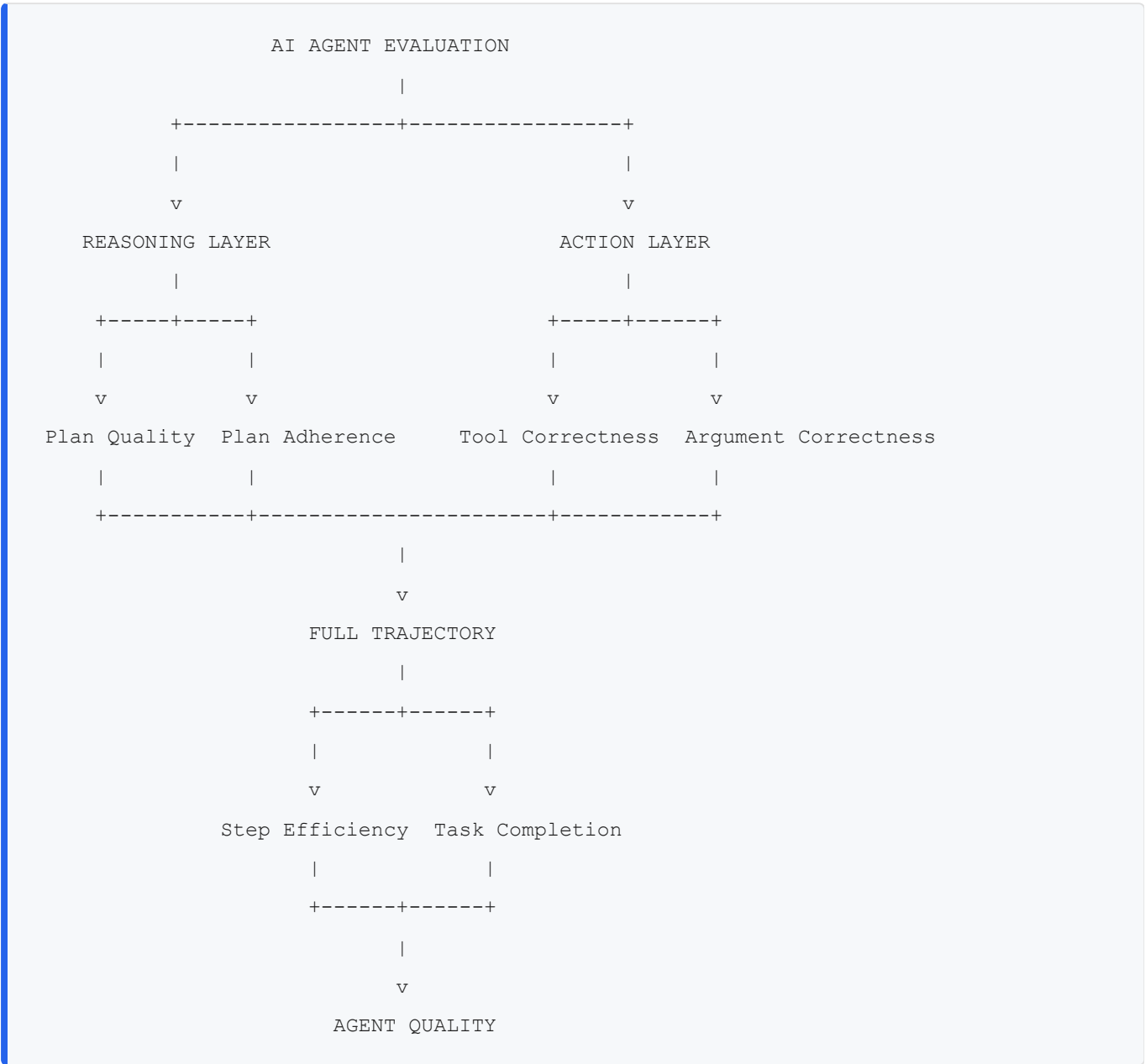
The supplied material also describes:

- Plan Quality
- Plan Adherence
- End-to-End evaluation
- Trajectory-based evaluation
- Component-level evaluation
- LLM-as-a-judge
- Hybrid evaluation
- System One / Jev-as-a-judge
- Dataset-based evaluation
- Tracing
- Thresholds
- Strict mode
- Reasons
- Verbose mode
- Component-level metric attachment

66. Final Reference Table

Evaluation Question	Metric
Is my agent's strategy logically good?	PlanQualityMetric
Did my agent execute the strategy it planned?	PlanAdherenceMetric
Did my agent choose the correct tool?	ToolCorrectnessMetric
Did my agent generate correct tool arguments?	ArgumentCorrectnessMetric
Did my agent avoid unnecessary actions?	StepEfficiencyMetric
Did my agent actually accomplish the user's task?	TaskCompletionMetric

The complete picture



Best practice: Use the metrics together to understand not only whether the agent succeeded, but also **why**, **how**, and **where** it succeeded or failed.